

Znajdowanie silnie spójnych składowych

Definicja.

Silnie spójną składową (sss) grafu skierowanego $G=(V,E)$ nazywamy każdy maksymalny w sensie inkluzji podgraf indukowany $G'=(V', E')$ taki, że dla każdej pary wierzchołków $u,v \in V'$ istnieje ścieżka z u do v .

Fakt.

Wierzchołki u, v są w tej samej sss wtedy i tylko wtedy gdy leżą na pewnym cyklu.

Wniosek:

Jeśli szukamy cyklu (lub listujemy cykle) wystarczy ograniczyć się do każdej sss oddzielnie.

Algorytm SSS (Kosaraju, Sharir, 1978-1981) - silnie spójne składowe:

We: $G=(V,E)$ - graf skierowany

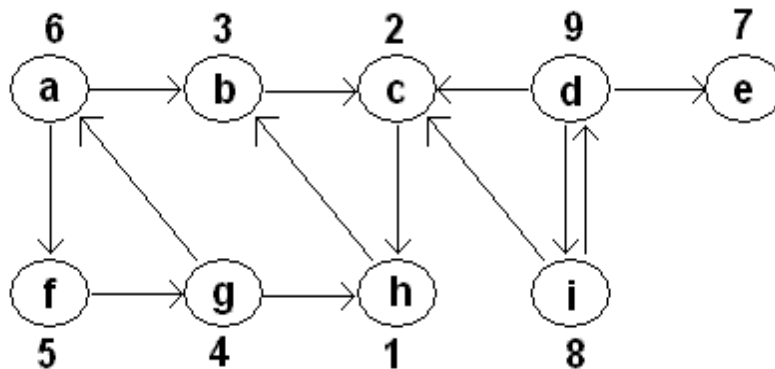
Wy: Etykieta $s(v)$, dla każdego wierzchołka v ,
identyfikująca silnie spójną składową
(tzn. $s(v)=s(u) \Leftrightarrow u,v$ leżą w jednej sss).

1. Algorytmem DFS dla grafu G oblicz czas $f(u)$ (postorder) dla każdego wierzchołka u ;
2. Oblicz G^T - graf transponowany (czyli wszystkie krawędzie skierowane w przeciwną stronę);
3. Wykonaj DFS(G^T), wywołując DFSVisit(u) w głównej pętli w kolejności malejących $f(u)$, wyliczonych w kroku 1; każde drzewo lasu rozpinającego wygenerowane w kroku 3 wypisz jako oddzielną silnie spójną składową.

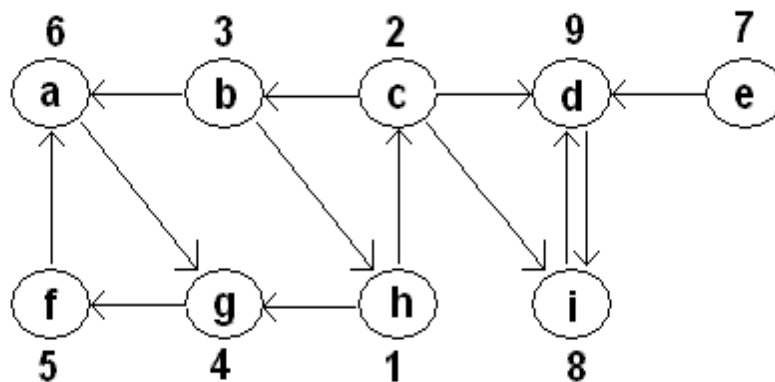
Praktycznie,

Krok 1. zamiast obliczać $f(u)$, w postorder kładziemy u na stos pomocniczy (jak w topsort);

Krok 3. W głównej pętli wywołujemy $\text{DFSVisit}(u)$ w kolejności jak na stosie pomocniczym.



Przykład: kolejność na listach sąsiedztwa alfabetyczna, obliczone wartości funkcji $f(v)$.



Graf transponowany. Korzenie drzew w kroku 3 i związane z nimi silnie spójne składowe wypisywane są w następującej kolejności (w preorder):

d : {d, i}

e : {e}

a : {a, g, f}

b : {b, h, c}

Poprawność algorytmu SSS

Definicja

Graf (silnie spójnych) składowych grafu skierowanego $G=(V,E)$ to graf ilorazowy względem relacji $R\subseteq V^2$, $uRv \Leftrightarrow u,v$ leżą na jednym cyklu.

Równoważnie, graf składowych:

$G^S = (V^S, E^S)$, gdzie

$V^S = \{C \subseteq V : C \text{ jest sss w } G\}$,

$C,D \in V^S, (C,D) \in E'$ jeśli istnieją $u \in C, v \in D$ takie że $(u,v) \in E$.

Fakt.

G^S jest DAG-iem.

Oczywiste.

Niech $f(u)$ = czas postorder wężła u obliczony przez $\text{DFS}(G)$ (krok 1 algorytmu SSS).

Dla $C \subseteq V$, niech $f(C) = \max\{f(u) : u \in C\}$.

Lemat 1.

Jeśli C, D - dwie różne silnie spójne składowe w G oraz istnieje $(u,v) \in E$, $u \in C, v \in D$ to $f(C) > f(D)$.

Dowód:

Podobnie jak w dowodzie poprawności DFS-topsort, z rozbiem na dwa przypadki - do której składowej weszliśmy wcześniej. QED

Lemat 2. (poprawność algorytmu SSS)

Kolejne drzewa generowane przez $\text{DFS}(G^T)$ w kroku 3 są silnie spójnymi składowymi.

Dowód: indukcja na kolejne drzewa.

Baza: zero drzew, OK.

$k > 0$:

wykażemy, że k -te drzewo T_k jest pewną nową sss C_k .

- drzewa $T_1, \dots, T_{k-1}$ są silnie spójnymi składowymi $C_1, \dots, C_{k-1}$, z założenia indukcyjnego;
- ponieważ sss w G^T są takie same jak w G , z korzenia wychodzą białe ścieżki do wszystkich wierzchołków składowej C_k . Zatem $C_k \subseteq T_k$;

Teraz wykazujemy, że w T_k nie znajdzie się żaden wierzchołek spoza C_k :

- składowe $C_1, \dots, C_{k-1}$ mogą być osiągalne z korzenia T_k , ale są już czarne, więc T_k ich nie obejmuje;
- jeśli C' jest składową różną od $C_1, \dots, C_k$, to $f(C') < f(C_k)$ - bo w takiej kolejności działa pętla główna w kroku 3;
- z Lematu 1 w grafie G nie istnieje krawędź z C' do C_k ;
- zatem w grafie G^T nie istnieje krawędź z C_k do C' ,
- a zatem T_k nie obejmuje żadnego wierzchołka z C' .

QED

Złożoność: $\Theta(n+m)$

Uwaga.

Najstarszy algorytm obliczania sss pochodzi od Tarjana (1971). Jest trudniejszy, zawiera jeden przebieg DFS-u ale jego idea jest bardziej uniwersalna.

Topsort - metoda 2: "obgryzanie grafu"

Bardziej intuicyjna niż DFS z numeracją postorder, przydatna w zastosowaniach.

Idea: jak w dowodzie, że każdy DAG da się posortować topologicznie.

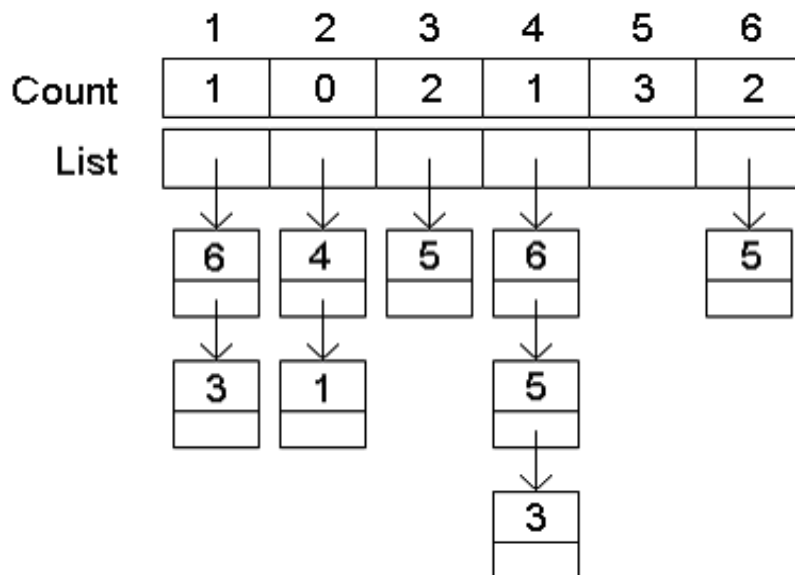
1. Znajdź wierzchołek u niemający poprzedników w G;
2. Wypisz u do ciągu wyjściowego i usuń u z grafu;
3. Powtarzaj 1,2 aż do wyczerpania zbioru wierzchołków.

Pomysły techniczne:

- dla każdego elementu przechowywana i uaktualniana liczba jego poprzedników,
- elementy dla których już nie ma poprzedników, a które jeszcze nie zostały wypisane na wyjście gromadzone są w kolejce lub dowolnym innym pojemniku,
- punkt 1 realizowany przez pobranie elementu z pojemnika, za wyjątkiem sytuacji początkowej, gdy realizowany jest przez przegląd całego grafu,
- podczas usuwania wierzchołka u usuwane są krawędzie z niego wychodzące;
- dla każdej takiej usuwanej krawędzi (u,v) aktualizowana jest liczba poprzedników wierzchołka v i jeśli się zeruje, to v jest dołączane do kolejki (zbiornika).

Przykład:

(4,3) , (1,3) , (2,1) , (3,5) , (4,5) , (2,4) , (4,6) , (6,5) , (1,6)



Poprzedników nie ma element 2. Usuń 2, pozostają:

(4,3) , (1,3) , (3,5) , (4,5) , (4,6) , (6,5) , (1,6)

Poprzedników nie mają 1 i 4. Usuń dowolny, np. 4:

(1,3) , (3,5) , (6,5) , (1,6)

Poprzedników nie ma 1. Usuń 1:

(3,5) , (6,5)

Poprzedników nie mają 3 i 6. Usuń np. 3:

(6,5)

Poprzedników nie ma 6. Usuń 6, zbiór par jest pusty, wypisz pozostałe elementy (tylko 5).

Wynik (elementy w kolejności wypisywania):

2, 4, 1, 3, 6, 5

Algorytm:

Struktury danych:

1. n - liczba elementów w zbiorze
2. m - liczba par relacji na wejściu do algorytmu
3. Listy następników dla każdego elementu. Komórki w listach są typu cell i zawierają pola
 succ // numer następnika danego elementu
 next // wskaźnik do następnego w liście
4. Tablica List[0.. $n-1$]
 List[i] - wskaźnik do początku listy
 następników elementu i , $i = 0, \dots, n-1$
5. Tablica Count[0.. $n-1$]
 Count[i] - liczba bezpośrednich poprzedników
 elementu i , $i \geq 0$
6. Q - zmienna typu kolejka

```
// inicjalizacja
for (i=0; i<n; i++)
    Count[i] := 0; List[i] := NULL;

// wczytywanie relacji i konstrukcja struktury
while (!end_of_file)
    wczytaj (i,j); Count[j] := Count[j] + 1;
    q := new cell; succ(q) := j;
    next(q) := List[i]; List[i] := q;
// komórkę zawierającą j dopisano na początek listy
// następników elementu i

// wyszukanie elementów bez poprzedników
// i zbudowanie z nich kolejki wyjściowej
Q.create;
for(i=0; i<n; i++)
    if (Count[i] = 0)
        Q.enqueue(i);
```

```

// usuń f - pierwszy z kolejki i wszystkie pary (f,j)
// jeśli dla usuwanej pary (f,j) następuje Count[j]=0,
// to j wstawiamy do kolejki
k := n;           // zliczamy ile elementów zostało
while (Q.nonempty) // początek kolejki wyjściowej
    f := Q.dequeue;
                    // usuń relacje dotyczące f
    p := List[f];
    while (p ≠ NULL)
        j := succ(p);
        Count[j] := Count[j] - 1;
        if (Count[j] = 0)
            Q.enqueue(j);
        p := next(p);

    write f; k := k-1;

// czy uporządkowano wszystkie elementy ?
if (k > 0) WYPISZ "Wśród pozostałych jest cykl";
END.

```

Złożoność: $\Theta(n+m)$.

Wersja Knuth'a (optymalizacja pamięci):

- kolejka wyjściowa tworzona w tablicy count[];
- jeśli count[i] staje się równe 0, to pole count[i] jest odtąd wykorzystane do linkowania kolejki;
- count[i]=k oznacza, że po elemencie i następny w kolejce jest element k,
- zaś count[i]=0 oznacza, że i jest ostatni w kolejce;
- utrzymujemy dodatkową zmienną wskazującą indeks ostatniego elementu w kolejce.